It's pretty straightforward. The syntax for named parameters is simply the variable name (without the dollar character) followed by a colon: `someFunction(paramName: $someData)`. The order does not matter, because the data can be mapped to the parameter's variable name. Prior to this feature's addition, functions that needed lots of parameters tended to have associative arrays passed in. The example above could be written like this:

```
function knowYourClassicMetal(array $data): void
{
    print "{$data['album']} by {$data['artist']}, released in
    {$data['year']} on {$data['label']}" . PHP_EOL;
}

knowYourClassicMetal([
    'album' => 'Somewhere in Time',
    'artist' => 'Iron Maiden',
    'year' => '1985',
    'label' => 'EMI'
]);
```

Which is fine as far as it goes, but (as well as fiddly-looking syntax) it suffers from a problem. It's not possible to know if all the required data is present in the incoming `$data` array without some checks and balances, and a missing array element will result in an `Undefined array key` error.

Code like that above should include some tests on the existence of array keys. With named parameters any missing data will emit a different `ArgumentCountError` but these are more easily spotted before run time, especially by an IDE (which will typically prompt for required arguments and/or highlight any that are missing as code is written).

While we're on the topic of associative arrays and named arguments, here's a nice additional feature of named arguments. If an associative array is unpacked into a function, and the keys are strings that match the parameter names, then the array values will be mapped as if they were named arguments. (Note: unpacking arrays that have string keys is only supported from PHP 8.0 upwards.)

```
$data = [
    'year' => '1995',
    'label' => 'Earache',
```